

Mini RFC: Advanced Transmission

April 5, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

In this mini RFC, we introduce several advanced transmission features as below:

- On the sending part of a stream, an application protocol can **reset** the stream (abrupt termination), resulting in a RESET_STREAM frame if the stream was not already in a terminal state.
- On the receiving part of a stream, an application protocol can **abort** reading of the stream and request closure, possibly resulting in a STOP_SENDING frame.
- QUIC allows for an arbitrary number of streams to operate concurrently. Streams can concurrently send data interleaved with other streams.

2 Resetting or Aborting a Stream

2.1 Resetting a Stream

Figure 1 shows the states for the part of a stream that sends data to a peer.

From any state that is one of “Ready”, “Send”, or “Data Sent”, an application can signal that it wishes to abandon transmission of stream data. Alternatively, an endpoint might receive a STOP_SENDING frame from its peer. In either case, the endpoint sends a RESET_STREAM frame, which causes the stream to enter the “Reset Sent” state.

We introduce the state machine in [mini RFC: Flow Control](#). Maybe I should find a better place for this topic, but at the moment, you should read §2.1 of [mini RFC: Flow Control](#) to learn the “normal” path of the state machine, i.e., without resetting or aborting.

An endpoint MAY send a RESET_STREAM as the first frame that mentions a stream; this causes the sending part of that stream to open and then immediately transition to the “Reset Sent” state.

Once a packet containing a RESET_STREAM has been acknowledged, the sending part of the stream enters the “Reset Recvd” state, which is a terminal state.

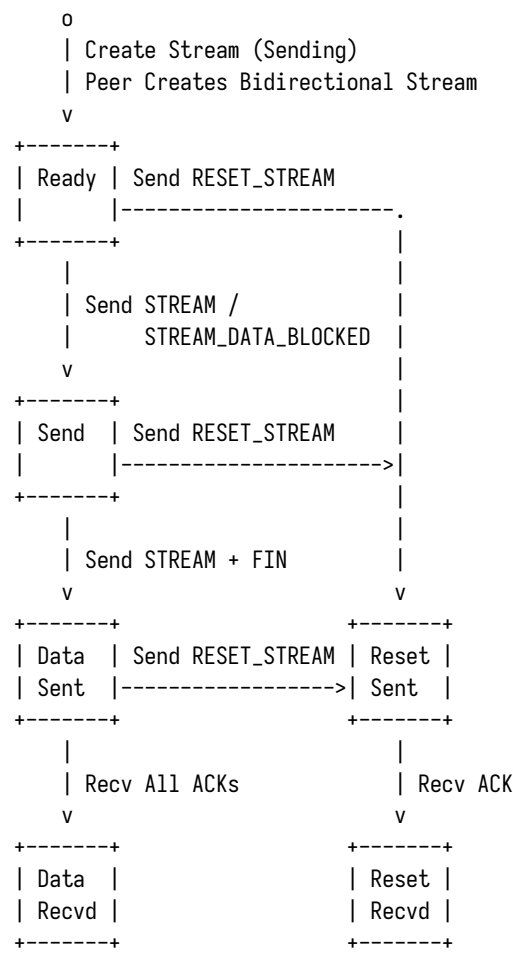


Figure 1: States for Sending Parts of Streams

Receiving a RESET_STREAM frame in the “Recv” or “Size Known” state causes the stream to enter the “Reset Recvd” state. This might cause the delivery of stream data to the application to be interrupted.

It is possible that all stream data has already been received when a RESET_STREAM is received (that is, in the “Data Recvd” state). Similarly, it is possible for remaining stream data to arrive after receiving a RESET_STREAM frame (the “Reset Recvd” state). An implementation is free to manage this

situation as it chooses.

Sending a RESET_STREAM means that an endpoint cannot guarantee delivery of stream data; however, there is no requirement that stream data not be delivered if a RESET_STREAM is received. An implementation MAY interrupt delivery of stream data, discard any data that was not consumed, and signal the receipt of the RESET_STREAM. A RESET_STREAM signal might be suppressed or withheld if stream data is completely received and is buffered to be read by the application. If the RESET_STREAM is suppressed, the receiving part of the stream remains in “Data Recvd”.

Once the application receives the signal indicating that the stream was reset, the receiving part of the stream transitions to the “Reset Read” state, which is a terminal state.

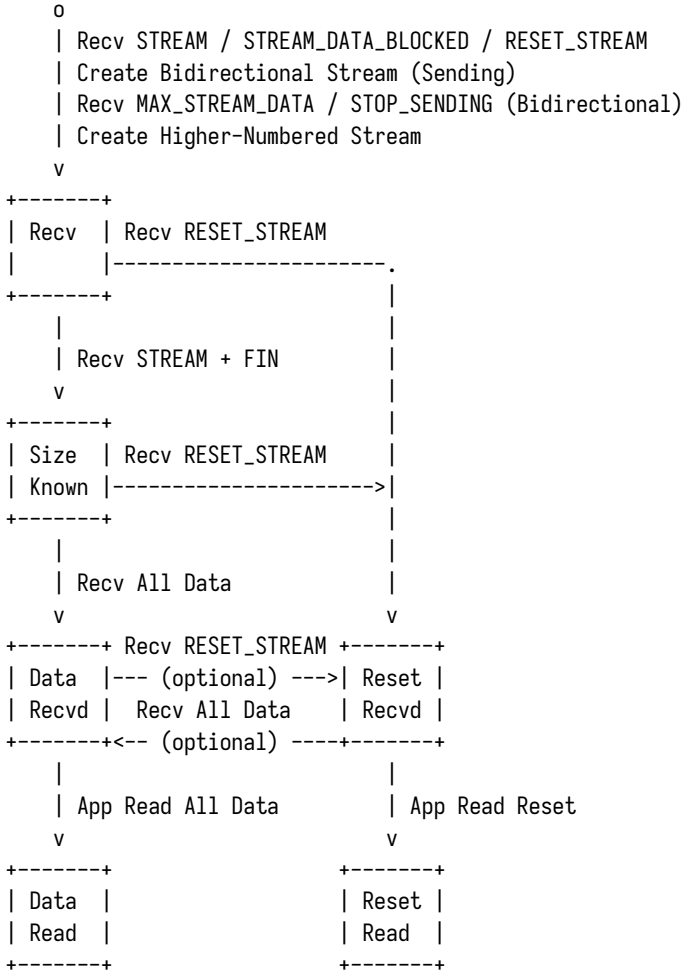


Figure 2: States for Receiving Parts of Streams

2.2 Permitted Frame Types

The sender of a stream sends just three frame types that affect the state of a stream at either the sender or the receiver: `STREAM`, `STREAM_DATA_BLOCKED`, and `RESET_STREAM`.

A sender **MUST NOT** send any of these frames from a terminal state (“Data Recvd” or “Reset Recvd”). A sender **MUST NOT** send a `STREAM` or `STREAM_DATA_BLOCKED` frame for a stream in the “Reset Sent” state or any terminal state – that is, after sending a `RESET_STREAM` frame. A receiver could receive any of these three frames in any state, due to the possibility of delayed delivery of packets carrying them.

The receiver of a stream sends `MAX_STREAM_DATA` and `STOP_SENDING` frames.

The receiver only sends `MAX_STREAM_DATA` frames in the “Recv” state. A receiver **MAY** send a `STOP_SENDING` frame in any state where it has not received a `RESET_STREAM` frame – that is, states other than “Reset Recvd” or “Reset Read”. However, there is little value in sending a `STOP_SENDING` frame in the “Data Recvd” state, as all stream data has been received. A sender could receive either of these two types of frames in any state as a result of delayed delivery of packets.

2.3 Solicited State Transition (Aborting a Stream)

If an application is no longer interested in the data it is receiving on a stream, it can abort reading the stream and specify an application error code.

If the stream is in the “Recv” or “Size Known” state, the transport **SHOULD** signal this by sending a `STOP_SENDING` frame to prompt closure of the stream in the opposite direction. This typically indicates that the receiving application is no longer reading data it receives from the stream, but it is not a guarantee that incoming data will be ignored.

`STREAM` frames received after sending a `STOP_SENDING` frame are still counted toward connection and stream flow control, even though these frames can be discarded upon receipt.

A `STOP_SENDING` frame requests that the receiving endpoint send a `RESET_STREAM` frame. An endpoint that receives a `STOP_SENDING` frame **MUST** send a `RESET_STREAM` frame if the stream is in the “Ready” or “Send” state. If the stream is in the “Data Sent” state, the endpoint **MAY** defer sending the `RESET_STREAM` frame until the packets containing outstanding data are acknowledged or declared lost. If any outstanding data is declared lost, the endpoint **SHOULD** send a `RESET_STREAM` frame instead of retransmitting the data.

An endpoint **SHOULD** copy the error code from the `STOP_SENDING` frame to the `RESET_STREAM` frame it sends, but it can use any application error code. An endpoint that sends a `STOP_SENDING` frame **MAY** ignore the error code in any `RESET_STREAM` frames subsequently received for that stream.

`STOP_SENDING` **SHOULD** only be sent for a stream that has not been reset by the peer. `STOP_SENDING` is most useful for streams in the “Recv” or “Size Known” state.

An endpoint is expected to send another `STOP_SENDING` frame if a packet containing a previous `STOP_SENDING` is lost. However, once either all stream data or a `RESET_STREAM` frame has been received for the stream – that is, the stream is in any state other than “Recv” or “Size Known” – sending a `STOP_SENDING` frame is unnecessary.

An endpoint that wishes to terminate both directions of a bidirectional stream can terminate one direction by sending a `RESET_STREAM` frame, and it can encourage prompt termination in the opposite direction by sending a `STOP_SENDING` frame.

3 Stream Multiplexing

Stream multiplexing is achieved by interleaving `STREAM` frames from multiple streams into one or more QUIC packets. A single QUIC packet can include multiple `STREAM` frames from one or more streams.

Stream multiplexing can have a significant effect on application performance if resources allocated to streams are correctly prioritized.

QUIC does not provide a mechanism for exchanging prioritization information. Instead, it relies on receiving priority information from the application.

A QUIC implementation **SHOULD** provide ways in which an application can indicate the relative priority of streams. An implementation uses information provided by the application to determine how to allocate resources to active streams.

4 Packets and Frames

4.1 Packets

[There is no new type of packets introduced in this RFC.](#)

4.2 Frames

4.2.1 `RESET_STREAM` Frames

An endpoint uses a `RESET_STREAM` frame (type=0x04) to abruptly terminate the sending part of a stream.

After sending a `RESET_STREAM`, an endpoint ceases transmission and retransmission of `STREAM` frames on the identified stream. A receiver of `RESET_STREAM` can discard any data that it already received on that stream.

An endpoint that receives a `RESET_STREAM` frame for a send-only stream **MUST terminate the connection with error `STREAM_STATE_ERROR`.**

`RESET_STREAM` frames are formatted as shown in [Figure 3](#).

```

RESET_STREAM Frame {
    Type (i) = 0x04,
    Stream ID (i),
    Application Protocol Error Code (i),
    Final Size (i),
}

```

Figure 3: *RESET_STREAM Frame Format*

RESET_STREAM frames contain the following fields:

- **Stream ID:** A variable-length integer encoding of the stream ID of the stream being terminated.
- **Application Protocol Error Code:** A variable-length integer containing the application protocol error code that indicates why the stream is being closed.
- **Final Size:** A variable-length integer indicating the final size of the stream by the RESET_STREAM sender, in units of bytes;

4.2.2 STOP_SENDING Frames

An endpoint uses a STOP_SENDING frame (type=0x05) to communicate that incoming data is being discarded on receipt per application request. STOP_SENDING requests that a peer cease transmission on a stream.

A STOP_SENDING frame can be sent for streams in the "Recv" or "Size Known" states. **Receiving a STOP_SENDING frame for a locally initiated stream that has not yet been created MUST be treated as a connection error of type STREAM_STATE_ERROR. An endpoint that receives a STOP_SENDING frame for a receive-only stream MUST terminate the connection with error STREAM_STATE_ERROR.**

STOP_SENDING frames are formatted as shown in **Figure 4**.

```

STOP_SENDING Frame {
    Type (i) = 0x05,
    Stream ID (i),
    Application Protocol Error Code (i),
}

```

Figure 4: *STOP_SENDING Frame Format*

STOP_SENDING frames contain the following fields:

- **Stream ID:** A variable-length integer carrying the stream ID of the stream being ignored.

- **Application Protocol Error Code:** A variable-length integer containing the application-specified reason the sender is ignoring the stream.

5 Application Protocol

5.1 Stream

This document defines a set of functions on streams that application protocols can rely upon.

- On the sending part of a stream, an application protocol can:
 - reset the stream (abrupt termination), resulting in a RESET_STREAM frame if the stream was not already in a terminal state. API: ResetStream
 - [get notified when the stream is aborted](#). API: SetStreamAbortedCallback
 - [Specify the priority of a stream when it's created](#). API: CreateStream
- On the receiving part of a stream, an application protocol can:
 - abort reading of the stream and request closure, possibly resulting in a STOP_SENDING frame. API: AbortStream
 - [get notified when the stream is reset](#). API: SetStreamResetCallback